DATA STRUCTURE - 7

C Programs with Questions and Sample Outputs

Prepared from the 10 questions in the uploaded handwritten notes.

Each question is followed by a complete C program and its sample output. Graph vertices are numbered from 0.

1. Expression Tree

Question:
Write a C program to construct an expression tree for the expression (a-b)+(c*d).

Program (C)

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    char data;
    struct Node *left, *right;
};

struct Node *newNode(char data) {
    struct Node *node = (struct Node *)malloc(sizeof(struct Node));
    node->data = data;
    node->left = node->right = NULL;
    return node;
}

void inorder(struct Node *root) {
    if (root != NULL) {
        if (root->left != NULL) printf("(");
        inorder(root->left);
        printf("%c", root->data);
        inorder(root->right);
        if (root->right != NULL) printf(")");
    }
}

void preorder(struct Node *root) {
    if (root != NULL) {
        printf("%c ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(struct Node *root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%c ", root->data);
    }
}

int main(void) {
    struct Node *root = newNode('+');
    root->left = newNode('-');
    root->right = newNode('*');
    root->left->left = newNode('a');
    root->left->right = newNode('b');
    root->right->left = newNode('c');
    root->right->right = newNode('d');

    printf("Infix expression: ");
    inorder(root);
    printf("\nPrefix: ");
    preorder(root);
    printf("\nPostfix: ");
    postorder(root);
    printf("\n");
    return 0;
}
```

Sample Output

```
Output:
Infix expression: (a-b)+(c*d)
Prefix: + - a b * c d
Postfix: a b - c d * +
```

2. Check Whether a Binary Tree Is a BST

Question:
Write a C program to check whether a binary tree is a Binary Search Tree (BST).

Program (C)

```c
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node *newNode(int value) {
    struct Node *node = (struct Node *)malloc(sizeof(struct Node));
    node->data = value;
    node->left = node->right = NULL;
    return node;
}

int isBST(struct Node *root, long long min, long long max) {
    if (root == NULL) return 1;
    if (root->data <= min || root->data >= max) return 0;

    return isBST(root->left, min, root->data) &&
           isBST(root->right, root->data, max);
}

int main(void) {
    struct Node *root = newNode(40);
    root->left = newNode(20);
    root->right = newNode(60);
    root->left->left = newNode(10);
    root->left->right = newNode(30);
    root->right->left = newNode(50);
    root->right->right = newNode(70);

    if (isBST(root, LLONG_MIN, LLONG_MAX))
        printf("The binary tree is a BST.\n");
    else
        printf("The binary tree is not a BST.\n");

    return 0;
}
```

Sample Output

```
Output:
The binary tree is a BST.
```

3. Implement a Max Heap

Question:
Write a C program to implement a max heap.

Program (C)

```
#include <stdio.h>

#define MAX 100

int heap[MAX];
int size = 0;

void insert(int value) {
    int i = size++;
    heap[i] = value;

    while (i > 0) {
        int parent = (i - 1) / 2;
        if (heap[parent] >= heap[i]) break;

        int temp = heap[parent];
        heap[parent] = heap[i];
        heap[i] = temp;
        i = parent;
    }
}

void display(void) {
    int i;
    for (i = 0; i < size; i++) {
        printf("%d ", heap[i]);
    }
    printf("\n");
}

int main(void) {
    insert(30);
    insert(10);
    insert(50);
    insert(20);
    insert(40);

    printf("Max heap array: ");
    display();
    printf("Maximum element: %d\n", heap[0]);
    return 0;
}
```

Sample Output

```
Output:
Max heap array: 50 40 30 10 20
Maximum element: 50
```

4. Detect a Cycle in an Undirected Graph

Question:
Given an undirected graph, write a C program to check whether it contains a cycle.

Program (C)

```
#include <stdio.h>

#define V 5

int graph[V][V] = {
    {0, 1, 1, 0, 0},
    {1, 0, 1, 0, 0},
    {1, 1, 0, 1, 0},
    {0, 0, 1, 0, 1},
    {0, 0, 0, 1, 0}
};

int dfs(int vertex, int visited[], int parent) {
    int i;
    visited[vertex] = 1;

    for (i = 0; i < V; i++) {
        if (graph[vertex][i]) {
            if (!visited[i]) {
                if (dfs(i, visited, vertex)) return 1;
            } else if (i != parent) {
                return 1;
            }
        }
    }
    return 0;
}

int main(void) {
    int visited[V] = {0};
    int i;

    for (i = 0; i < V; i++) {
        if (!visited[i] && dfs(i, visited, -1)) {
            printf("The graph contains a cycle.\n");
            return 0;
        }
    }

    printf("The graph does not contain a cycle.\n");
    return 0;
}
```

Sample Output

```
Output:
The graph contains a cycle.
```

5. Count Number of Islands

Question:
Given a grid containing '1' (land) and '0' (water), write a C program to count the number of islands. Land cells connected horizontally or vertically belong to the same island.

Program (C)

```
#include <stdio.h>

#define ROW 5
#define COL 5

char grid[ROW][COL] = {
    {'1','1','0','0','0'},
    {'1','1','0','0','1'},
    {'0','0','1','0','1'},
    {'0','0','0','1','1'},
    {'1','0','0','0','0'}
};

void dfs(int r, int c) {
    if (r < 0 || r >= ROW || c < 0 || c >= COL ||
        grid[r][c] != '1') {
        return;
    }

    grid[r][c] = '0';
    dfs(r - 1, c);
    dfs(r + 1, c);
    dfs(r, c - 1);
    dfs(r, c + 1);
}

int main(void) {
    int r, c, islands = 0;

    for (r = 0; r < ROW; r++) {
        for (c = 0; c < COL; c++) {
            if (grid[r][c] == '1') {
                islands++;
                dfs(r, c);
            }
        }
    }

    printf("Number of islands: %d\n", islands);
    return 0;
}
```

Sample Output

```
Output:
Number of islands: 4
```

6. Minimum Spanning Tree Using Kruskal's Algorithm

Question:
Write a C program to find the minimum spanning tree (MST) using Kruskal's algorithm.

Program (C)

```c
#include <stdio.h>
#include <stdlib.h>

#define V 4
#define E 5

struct Edge {
    int src, dest, weight;
};

int parent[V];

int compare(const void *a, const void *b) {
    const struct Edge *x = (const struct Edge *)a;
    const struct Edge *y = (const struct Edge *)b;
    return x->weight - y->weight;
}

int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]);
    return parent[x];
}

void unite(int a, int b) {
    parent[find(a)] = find(b);
}

int main(void) {
    struct Edge edges[E] = {
        {0, 1, 10}, {0, 2, 6}, {0, 3, 5},
        {1, 3, 15}, {2, 3, 4}
    };
    int i, count = 0, total = 0;

    for (i = 0; i < V; i++) parent[i] = i;
    qsort(edges, E, sizeof(edges[0]), compare);

    printf("Edges in the MST:\n");
    for (i = 0; i < E && count < V - 1; i++) {
        int u = edges[i].src;
        int v = edges[i].dest;

        if (find(u) != find(v)) {
            printf("%d - %d : %d\n", u, v, edges[i].weight);
            unite(u, v);
            total += edges[i].weight;
            count++;
        }
    }

    printf("Total MST weight: %d\n", total);
    return 0;
}
```

Sample Output

```
Output:
Edges in the MST:
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10
Total MST weight: 19
```

7. Dijkstra's Shortest-Path Algorithm

Question:
Write a C program to find the shortest paths from a source vertex to all other vertices using Dijkstra's algorithm. Assume all edge weights are non-negative.

Program (C)

```
#include <stdio.h>

#define V 5
#define INF 99999

int graph[V][V] = {
    {0, 10, 0, 5, 0},
    {0, 0, 1, 2, 0},
    {0, 0, 0, 0, 4},
    {0, 3, 9, 0, 2},
    {7, 0, 6, 0, 0}
};

int main(void) {
    int dist[V], visited[V] = {0};
    int i, j, count, min, u, source = 0;

    for (i = 0; i < V; i++) dist[i] = INF;
    dist[source] = 0;

    for (count = 0; count < V - 1; count++) {
        min = INF;
        u = -1;

        for (i = 0; i < V; i++) {
            if (!visited[i] && dist[i] < min) {
                min = dist[i];
                u = i;
            }
        }

        if (u == -1) break;
        visited[u] = 1;

        for (j = 0; j < V; j++) {
            if (!visited[j] && graph[u][j] > 0 &&
                dist[u] + graph[u][j] < dist[j]) {
                dist[j] = dist[u] + graph[u][j];
            }
        }
    }

    printf("Shortest distances from source %d:\n", source);
    for (i = 0; i < V; i++) {
        if (dist[i] == INF) printf("To %d: unreachable\n", i);
        else printf("To %d: %d\n", i, dist[i]);
    }
    return 0;
}
```

Sample Output

```
Output:
Shortest distances from source 0:
To 0: 0
To 1: 8
To 2: 9
To 3: 5
To 4: 7
```

8. Find a Mother Vertex

Question:
Write a C program to find a mother vertex in a directed graph. A mother vertex can reach every other vertex.

Program (C)

```
#include <stdio.h>

#define V 5

int graph[V][V] = {
    {0, 1, 1, 0, 0},
    {0, 0, 0, 1, 0},
    {0, 0, 0, 0, 1},
    {0, 0, 0, 0, 0},
    {0, 0, 0, 1, 0}
};

void dfs(int v, int visited[]) {
    int i;
    visited[v] = 1;
    for (i = 0; i < V; i++) {
        if (graph[v][i] && !visited[i]) dfs(i, visited);
    }
}

int main(void) {
    int visited[V] = {0};
    int i, candidate = -1;

    for (i = 0; i < V; i++) {
        if (!visited[i]) {
            dfs(i, visited);
            candidate = i;
        }
    }

    for (i = 0; i < V; i++) visited[i] = 0;
    dfs(candidate, visited);

    for (i = 0; i < V; i++) {
        if (!visited[i]) {
            printf("No mother vertex exists.\n");
            return 0;
        }
    }

    printf("Mother vertex: %d\n", candidate);
    return 0;
}
```

Sample Output

```
Output:
Mother vertex: 0
```

9. Find All Strongly Connected Components (SCCs)

Question:
Given a directed graph, write a C program to print all strongly connected components (SCCs) using Kosaraju's algorithm.

Program (C)

```
#include <stdio.h>

#define V 5

int graph[V][V] = {
    {0, 1, 0, 0, 0},
    {0, 0, 1, 0, 0},
    {1, 0, 0, 1, 0},
    {0, 0, 0, 0, 1},
    {0, 0, 0, 1, 0}
};

void dfs1(int v, int visited[], int stack[], int *top) {
    int i;
    visited[v] = 1;
    for (i = 0; i < V; i++) {
        if (graph[v][i] && !visited[i])
            dfs1(i, visited, stack, top);
    }
    stack[++(*top)] = v;
}

void dfs2(int v, int visited[], int transpose[V][V]) {
    int i;
    visited[v] = 1;
    printf("%d ", v);
    for (i = 0; i < V; i++) {
        if (transpose[v][i] && !visited[i])
            dfs2(i, visited, transpose);
    }
}

int main(void) {
    int transpose[V][V], visited[V] = {0};
    int stack[V], top = -1, i, j;

    for (i = 0; i < V; i++)
        for (j = 0; j < V; j++)
            transpose[i][j] = graph[j][i];

    for (i = 0; i < V; i++)
        if (!visited[i]) dfs1(i, visited, stack, &top);

    for (i = 0; i < V; i++) visited[i] = 0;

    printf("Strongly connected components:\n");
    while (top >= 0) {
        int v = stack[top--];
        if (!visited[v]) {
            dfs2(v, visited, transpose);
            printf("\n");
        }
    }
    return 0;
}
```

Sample Output

```
Output:
Strongly connected components:
0 2 1
3 4
```

10. Check Whether a Graph Is Bipartite

Question:
Write a C program to determine whether an undirected graph is bipartite. A bipartite graph's vertices can be divided into two sets so that no edge joins vertices in the same set.

Program (C)

```
#include <stdio.h>

#define V 4

int graph[V][V] = {
    {0, 1, 1, 0},
    {1, 0, 0, 1},
    {1, 0, 0, 1},
    {0, 1, 1, 0}
};

int isBipartite(void) {
    int color[V], queue[V];
    int i, front, rear, u, v;

    for (i = 0; i < V; i++) color[i] = -1;

    for (i = 0; i < V; i++) {
        if (color[i] != -1) continue;

        front = rear = 0;
        queue[rear++] = i;
        color[i] = 0;

        while (front < rear) {
            u = queue[front++];

            for (v = 0; v < V; v++) {
                if (!graph[u][v]) continue;

                if (color[v] == -1) {
                    color[v] = 1 - color[u];
                    queue[rear++] = v;
                } else if (color[v] == color[u]) {
                    return 0;
                }
            }
        }
    }
    return 1;
}

int main(void) {
    if (isBipartite())
        printf("The graph is bipartite.\n");
    else
        printf("The graph is not bipartite.\n");
    return 0;
}
```

Sample Output

```
Output:
The graph is bipartite.
```